Group 21 Progress Report: Instrument Cataloguer

Emre Bozkurt, Thanihan Parasutharan, Yashasvi Seth
{bozkurte,parasutt,sethy}@mcmaster.ca

1 Introduction

Musicians, producers, and archivists work with large collections of recorded audio that are often poorly labelled. Sample libraries can contain thousands of unnamed or inconsistently named clips, and searching recordings by instrument usually depends on someone having tagged them by hand. A model that identifies the instrument from the sound alone would allow these collections to be tagged and searched automatically, and it is a first step towards recognizing instruments within full pieces of music.

We are creating a program that catalogues orchestral instruments based on their waveform. This is a single-label, multi-class classification task with 19 classes, one per brass, woodwind, and string instrument, where each clip is one note by one instrument from the Philharmonia Orchestra sound sample library (Philharmonia Orchestra, n.d.). The raw waveform is converted with a Fourier transform into a time–frequency representation, from which we either build hand-crafted features for a support vector machine (SVM) baseline or compute log-mel spectrograms for a small convolutional neural network (CNN), which is the main goal of the project.

This task is challenging for several reasons. Pitch can act as a shortcut, since low instruments such as the tuba, contrabassoon, and double bass can be partly identified by register alone, so the model must learn timbre rather than pitch. Instruments in the same family, such as viola and violin, sound very alike. The classes are imbalanced, from 74 clips (banjo) to 1,501 (violin). The dataset contains near-identical recordings of the same note at different dynamics and durations, so a random train/test split overstates performance. Finally, the studio recordings may not transfer to noisier real-world audio.

2 Related Work

Early instrument recognition work focused on isolated notes, as in our task, using hand-crafted features. Eronen and Klapuri (2000) combined cepstral coefficients with temporal features describing a note's attack and decay, and Herrera-Boyer et al. (2003) surveyed spectral and temporal features and classifiers such as k-nearest neighbours, decision trees, and SVMs. These works motivate our SVM baseline. More recent work uses larger datasets and deep learning. NSynth (Engel et al., 2017) contains over 300,000 notes labelled by instrument family, and IRMAS (Bosch et al., 2012) targets the harder problem of predominant instrument recognition in polyphonic music, where Han et al. (2017) showed that a CNN on mel spectrograms outperforms feature-based approaches. This motivates our log-mel CNN. PANNs (Kong et al., 2020), CNNs pretrained on large-scale audio data, are a pretrained alternative if our dataset proves too small.

Two existing solutions use the same Philharmonia dataset. Mahanta et al. (2021) trained a fully connected network on MFCCs and reported 97% accuracy, but on a random 80/20 split, which leaks near-duplicate notes into the test set, and without pitch-shortcut or noise testing. Siebert (2016) found that hand-crafted descriptors with k-nearest neighbours and SVMs do well on Philharmonia but noticeably worse on a different recording set. We differ from both by using a grouped split and by explicitly testing for pitch shortcuts and noise robustness.

3 Dataset

We use the Philharmonia Orchestra sound sample library (Philharmonia Orchestra, n.d.), distributed as MP3 files whose names record the instrument, pitch, duration, dynamic, and articulation (e.g., `flute_A4_025_forte_normal.mp3`). The instru-

Instrument	Clips	Pitch range
Brass		
French horn	652	A♯1–F5
Trombone	831	E2–F6
Trumpet	485	E2–E6
Tuba	972	A♯0–F4
Woodwind		
Bass clarinet	944	C2–C6
Bassoon	720	A♯1–G5
Clarinet	846	D3–C7
Contrabassoon	710	A♯0–D4
Cor anglais	691	E3–B5
Flute	878	C4–F7
Oboe	595	A♯3–A♯6
Saxophone	732	D3–G6
Strings		
Banjo	74	C3–E6
Cello	889	C2–C7
Double bass	852	C1–G4
Guitar	106	E2–E6
Mandolin	80	G3–A6
Viola	972	C3–F♯7
Violin	1,501	G3–E8
Total	**13,530**	

Table 1: Number of clips and pitch range for each instrument, after removing duplicate recordings and one unreadable file.

ment serves as our label, and the other fields let us analyse results by pitch and dynamic without manual annotation. We use the brass, string, and woodwind sections (Table 1), leaving out percussion because those instruments do not have many recordings.

Our proposal counted 14,226 clips, but the cor anglais folder contains every recording twice, also under its alternate name `english-horn`. We remove these 691 byte-identical duplicates, plus four more found by hashing every file, so that copies of a recording cannot appear in both training and test. One viola file could not be decoded, which leaves 13,530 clips. Most are single notes of 0.25 to 1.5 seconds, with some longer sustained notes and short phrases, dynamics from pianissimo to fortissimo, and about 84% in the standard articulation. The classes are highly imbalanced: violin has about twenty times as many clips as banjo, and the three plucked strings make up less than 2% of the data.

4 Features

Each MP3 is decoded, converted to mono, resampled to 16 kHz, trimmed of silence, and padded or cropped to 2 seconds. We compute a short-time Fourier transform, map it onto the mel scale, which approximates human pitch perception, and take the logarithm, giving a log-mel spectrogram.

SVM features. Since the SVM needs a fixed-length input, we summarize each frame-level feature by its mean and standard deviation over time. Following Eronen and Klapuri (2000) and Herrera-Boyer et al. (2003), we use 20 MFCCs, which describe the overall shape of the spectrum and are the main indicator of timbre; spectral centroid and rolloff, which measure brightness; spectral flatness and zero-crossing rate, which measure noisiness; and attack time, the time the energy envelope takes to reach its peak. This gives 49 dimensions, which we standardize using training set statistics. Instead of automatic feature selection, we measure each feature's contribution by ablation (Section 6).

librosa. All features are computed with librosa (McFee et al., 2015). They involve no learned models, only deterministic signal processing; MFCCs, for example, apply a mel filterbank, a logarithm, and a discrete cosine transform to each 2048-sample frame. Their reliability therefore depends on the parameters chosen. One consequence is that attack time is only measured to the nearest 32 ms frame, which is coarse for fast attacks.

CNN input. The CNN takes the log-mel spectrogram directly as a single-channel image and is trained end-to-end with no hand-crafted features. To reduce overfitting, we will apply SpecAugment-style masking (Park et al., 2019), which hides random frequency bands and time steps during training.

5 Implementation

We implement a majority-class baseline, an SVM, and a CNN, using librosa, scikit-learn, and PyTorch. Both baselines are implemented and evaluated, and the CNN is designed but still being implemented. The majority baseline always predicts violin, giving about 11% accuracy but only 5.3% balanced accuracy and 1% macro-F1, which shows why accuracy alone is misleading here.

SVM. Following feedback on our proposal, we use a single classical baseline. We chose an SVM over a random forest because our features are dense and standardized, which suits a kernel method, and because SVMs were among the strongest classifiers for isolated notes in Herrera-Boyer et al. (2003). We use an RBF kernel $k(\mathbf{x}, \mathbf{x}') = \exp(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2)$ and minimize the class-weighted soft-margin

Layer	Output channels
Input log-mel spectrogram	1
Conv block 1	32
Conv block 2	64
Conv block 3	128
Conv block 4	256
Global average pooling	256
Dropout ($p = 0.3$) + linear	19

Table 2: Planned CNN architecture. Each conv block is a 3×3 convolution, batch normalization, ReLU, and 2×2 max pooling.

hinge loss

$$\frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{N} w_{y_i} \max\left(0,\, 1 - t_i f(\mathbf{x}_i)\right), \quad (1)$$

where $t_i \in \{-1, +1\}$ is the binary label and w_{y_i} is inversely proportional to the frequency of class y_i, so errors on rare instruments cost more. scikit-learn solves this in its dual form, handling the 19 classes one-vs-one. A grid search over $C \in \{0.1, 1, 10, 100\}$ and $\gamma \in \{10^{-3}, 10^{-2}, 10^{-1}, 1/49\}$ on the validation set selected $C = 10$ and $\gamma = 0.01$ (validation macro-F1 90.8%).

CNN. Our main model has four convolutional blocks (Table 2), each a 3×3 convolution, batch normalization, ReLU, and 2×2 max pooling. Global average pooling then makes the network independent of clip length, and dropout and a linear layer produce 19 class scores, about 0.4 million parameters in total. We will train it with the class-weighted cross-entropy loss

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N} w_{y_i} \log \hat{p}_{i,y_i}, \qquad w_c = \frac{N_{\text{train}}}{C \cdot N_c}, \quad (2)$$

where N_c is the number of training clips in class c, using Adam (learning rate 10^{-3}, batch size 64) with early stopping on validation macro-F1.

Implementation details. We decode the audio once and cache features as NumPy arrays, so every model sees the same inputs, and we fix random seeds for reproducibility. Feature extraction for all clips takes under a minute on a 22-core machine. Our main practical problems were the duplicate recordings, found only because we hash every file before splitting, and one undecodable file, which the loader now skips and reports instead of crashing.

Model	Acc.	Bal. acc.	Macro-F1
Majority class	11.0	5.3	1.0
SVM, random split	98.6	98.6	98.7
SVM, grouped split	90.6	90.8	90.4
CNN, grouped split	–	–	–

Table 3: Test set results (%). The grouped split keeps all clips of an instrument and pitch in the same split. The CNN is still being implemented.

6 Evaluation and Results

Splits. As described in our proposal, we use a grouped split: all clips of a given (instrument, pitch) pair go into the same split, so the same note at different dynamics or durations never appears in both training and test. We use stratified group k-fold with seven folds, giving about 70% training, 15% validation, and 15% test with class proportions close to the full dataset. To measure how much a naive split overstates performance, we also report a random clip-level split, as used by Mahanta et al. (2021). Hyperparameters and features are chosen on the validation set, and the test set is used once. We use a fixed split rather than cross-validation so that the CNN, which is expensive to train, is evaluated on the same data as the baselines.

Metrics. We use macro-F1 and balanced accuracy, since the dataset is imbalanced, and report accuracy only for reference. As our TA advised at the proposal stage, we also analyse each class individually with per-class F1 and a confusion matrix, since a large class such as violin could inflate overall accuracy.

Results so far. On the random split, the SVM reaches 98.7% macro-F1, close to the 97% of Mahanta et al. (2021). On the grouped split, it drops to 90.4% (Table 3), confirming the concern from our proposal that a random split overstates performance, here by about 8 points. The errors match the similar-timbre challenge. The weakest classes are viola (76.1% F1), oboe (76.7%), and clarinet (80.0%), and the most common confusions are within families: viola as violin (28 clips), clarinet as saxophone (13), cello as viola (11), and oboe as clarinet (10). Meanwhile, tuba, double bass, and contrabassoon score 98–100%, suggesting the model uses their register as a shortcut. Class weighting keeps the rare plucked strings at 89–97% F1.

Feature ablation. Following our TA's suggestion, we retrained the SVM with features removed,

Feature set	Val. macro-F1
All features (20 MFCCs)	90.8
without MFCCs	55.9
without one spectral feature	90.0–90.5
without attack time	90.3
MFCCs only	88.7
best single non-MFCC feature	17.5
MFCC means only	88.1
MFCC standard deviations only	79.0
5 / 10 / 13 MFCCs (others kept)	91.4 / **93.9** / 93.8

Table 4: SVM feature ablation on the grouped split ($C = 10$, γ scaled by feature dimension). The spectral features are centroid, rolloff, flatness, and zero-crossing rate.

scaling γ by the feature dimension and scoring on the validation set (Table 4). The MFCCs dominate: without them macro-F1 falls from 90.8% to 55.9%, while every other feature alone scores below 18%. The other features each add under one point, and greedy backward elimination kept all of them. MFCC means are more useful than standard deviations, so the overall spectral shape matters more than its change during the note. Most usefully, keeping only 10 MFCCs raises macro-F1 to 93.9%. We believe the higher coefficients capture fine harmonic detail that depends on the note being played, which hurts generalization to the unseen pitches of the grouped split. We will use 10 MFCCs and report the re-tuned model's test score once, alongside the CNN.

7 Feedback and Plans

Feedback. The feedback on our proposal suggested focusing on a single classical baseline, so we dropped the random forest and measure feature contributions by ablation instead of its feature importances. At the proposal stage, our TA, Abdulazeez, felt the project was challenging but appropriate, and noted that turning audio into spectrograms makes it a vision-style problem, which confirmed our choice of a CNN. He advised us to measure differences between the spectrograms of different instruments and to analyse each class individually.

In our progress meeting, our TA, Hongyi Zhang, made two suggestions. First, he recommended ablations that leave out individual features to optimize the feature set. This led to our most useful improvement so far, raising validation macro-F1 by three points with 10 MFCCs. Second, he suggested checking how accurately librosa extracts features and whether it relies on trustworthy models. Since librosa's features are deterministic computations rather than learned models, the main risk is incorrect parameters. We will verify them on synthetic signals with known features (a pure sine wave's spectral centroid should equal its frequency, and white noise should have a flatness near 1), and compare our MFCCs against torchaudio's.

Next steps. Our priority is to finish the CNN and compare it with the SVM on the grouped split. We then plan to:

- re-tune the SVM with 10 MFCCs and try MFCC deltas;
- validate librosa's features as described above;
- run CNN ablations without class weighting, without SpecAugment, and with 1-second clips;
- compare against a pitch-only baseline and measure accuracy on the pitch range where instruments overlap;
- compute distances between the average log-mel spectrograms of each instrument, to help explain the confusions;
- try PANNs embeddings if the small CNN underperforms;
- add noise to the test data at increasing levels and plot accuracy against signal-to-noise ratio.

Reflection. Our results confirm the evaluation concerns from our proposal: the random split made a simple SVM look nearly perfect, and only the grouped split revealed the expected within-family confusions. Rather than improving on a near-perfect score, the CNN's job is to reduce confusions between similar instruments such as viola and violin, which hand-crafted summary statistics struggle to separate.

Team Contributions

Emre, *bozkurte*, worked on the SVM baseline and evaluation, Thanihan, *parasutt*, worked on the CNN architecture and feature extraction, and Yash, *sethy*, worked on the parsing the dataset and feature ablation. All three contributed to the proposal and progress report.

References

Juan J. Bosch, Jordi Janer, Ferdinand Fuhrmann, and Perfecto Herrera. 2012. A comparison of sound segregation techniques for predominant instrument recognition in musical audio signals. In *Proceedings of the 13th International Society for Music Information Retrieval Conference*.

Jesse Engel, Cinjon Resnick, Adam Roberts, Sander Dieleman, Douglas Eck, Karen Simonyan, and Mohammad Norouzi. 2017. Neural audio synthesis of musical notes with WaveNet autoencoders. In *Proceedings of the 34th International Conference on Machine Learning*.

Antti Eronen and Anssi Klapuri. 2000. Musical instrument recognition using cepstral coefficients and temporal features. In *Proceedings of the IEEE International Conference on Acoustics, Speech, and Signal Processing*.

Yoonchang Han, Jaehun Kim, and Kyogu Lee. 2017. Deep convolutional neural networks for predominant instrument recognition in polyphonic music. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 25(1):208–221.

Perfecto Herrera-Boyer, Geoffroy Peeters, and Shlomo Dubnov. 2003. Automatic classification of musical instrument sounds. *Journal of New Music Research*, 32(1):3–21.

Qiuqiang Kong, Yin Cao, Turab Iqbal, Yuxuan Wang, Wenwu Wang, and Mark D. Plumbley. 2020. PANNs: Large-scale pretrained audio neural networks for audio pattern recognition. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 28:2880–2894.

Saranga Kingkor Mahanta, Abdullah Faiz Ur Rahman Khilji, and Partha Pakray. 2021. Deep neural network for musical instrument recognition using MFCCs. *Computación y Sistemas*, 25(2):351–360.

Brian McFee, Colin Raffel, Dawen Liang, Daniel P. W. Ellis, Matt McVicar, Eric Battenberg, and Oriol Nieto. 2015. librosa: Audio and music signal analysis in python. In *Proceedings of the 14th Python in Science Conference*, pages 18–25.

Daniel S. Park, William Chan, Yu Zhang, Chung-Cheng Chiu, Barret Zoph, Ekin D. Cubuk, and Quoc V. Le. 2019. SpecAugment: A simple data augmentation method for automatic speech recognition. In *Proceedings of Interspeech*, pages 2613–2617.

Philharmonia Orchestra. n.d. Sound samples. https://philharmonia.co.uk/resources/sound-samples/. Accessed: 2026-11-02.

Xavier Siebert. 2016. Study of the robustness of descriptors for musical instruments classification. In *Proceedings of Benelearn 2016*.